SPRINT 10 · REDES NEURONALES · S10.2

Redes densas con Keras

Con Keras montamos el mismo tipo de red en pocas líneas y la aplicamos a las reservas de TurisData: clasificar cancelaciones y predecir el precio.

Preparar los datos para una red

Las redes son **sensibles a la escala**: si `antelacion_dias` va de 0 a 400 y `adultos` de 1 a 4, la primera «grita» más.

```
cols_num = ["antelacion_dias", "noches", "adultos", "ninos", "peticiones_especiales",
            "cliente_repetidor", "cancelaciones_previas", "mes_llegada"]    # Ejercicio 1
cols_cat = ["tipo_habitacion", "regimen", "canal", "pais_origen", "tarifa", "deposito"]
```

- Numéricas: **estandarizar**. Categóricas: **one-hot**.
- Sin `id_reserva`, sin fechas y sin el objetivo.

Un MLP para clasificar cancelaciones

Pila de capas `Dense`: todas las neuronas de una capa conectadas con todas las de la siguiente.

```
def construir_mlp_clasificacion(n_entradas, capas=(32, 16)):          # Ejercicio 2
    m = keras.Sequential([keras.layers.Input((n_entradas,))])
    for u in capas:
        m.add(keras.layers.Dense(u, activation="relu"))
    m.add(keras.layers.Dense(1, activation="sigmoid"))
    m.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=[keras.metrics.AUC(name="auc")])
    return m
```

Ocultas: `relu`. Salida: 1 neurona `sigmoid` (probabilidad de cancelar).
Pérdida: `binary_crossentropy`.

Entrenar y comparar con la logística

```
hist = modelo.fit(A_ent, y_ent, epochs=15, batch_size=64,
                  validation_data=(A_val, y_val))
auc_mlp = roc_auc_score(yc_pru, prob_mlp)                    # Ejercicio 3
auc_log = roc_auc_score(yc_pru, prob_log)
mlp_gana = auc_mlp > auc_log + 0.02
```

- **Época**: una pasada por todos los datos; **lote** (`batch_size`): filas por actualización.
- ¿El MLP gana por **más de 0,02 de AUC**? Esa respuesta es la pregunta del reto en pequeño. Con datos tabulares, no siempre.

Un MLP para predecir el precio (regresión)

	Clasificación	Regresión
Salida	`Dense(1, "sigmoid")`	`Dense(1)` **sin activación**
Pérdida	`binary_crossentropy`	`mse`
Métrica legible	AUC	`mae` (euros)

```
mae_mlp_euros = mean_absolute_error(yr_pru, pred_mlp)          # Ejercicio 5
mejora_vs_media = 1 - mae_mlp_euros / mae_base
```

El objetivo (`precio_noche`) se **escala** para entrenar y la predicción se devuelve a euros. `construir_mlp_regresion` es el Ejercicio 4.

Mini-reto: ¿es mejor la red grande?

```
hist_ancho = construir_mlp_clasificacion(n, capas=(128, 64)).fit(
    A_ent, y_ent, epochs=15, batch_size=64, validation_data=(A_val, y_val))
auc_ancho = hist_ancho.history["val_auc"][-1]
```

Más neuronas = más capacidad… y más facilidad para **memorizar**. Compara `auc_ancho` con el MLP pequeño y escribe tu conclusión. Eso lleva a la siguiente presentación: regularización.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S10_02_mlp_keras — 60 min

MLP en Keras: clasificación y regresión

sprints/S10/notebooks/S10_02_mlp_keras.ipynb